

Concurrency and Synchronization

Development environment

- concurrent programming (using Processes or Threads)
- cooperating processes or threads

Issues:

- Need to manipulate shared data
- Race conditions may arise
- There may be sections of not-reentrant code (non interruptible)

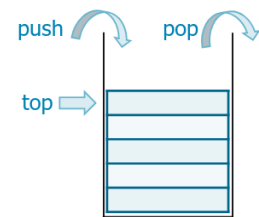
Solution strategy

- appropriately synchronize P or T, to make their results not dependent on their relative speed

Example:

Time	Person A	Person B
10.00	Look in fridge; out of milk	
10.05	Leave for store	
10.10	Get to the store	Look in fridge; out of milk
10.15	Buy milk	Leave for store
10.20	Back home	Get to the store
10.25	Put milk in fridge	Buy milk
10.30		Back home
10.35		Put milk in fridge Hops!!!

LIFO - Stack



```
void push (int val) {
    if(top>=SIZE) return;
    stack[top] = val;
    top++;
    return;
}

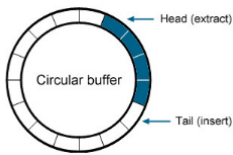
int pop (int *val) {
    if(top<=0) return;
    top--;
    *val=stack[top];
    return;
}
```

push and pop

- Operate on the same end of the stack
- Variable `top` is shared

The problem is that we can overwrite a value or make a pop of a nonexistent value.

Circular buffer



```
void enqueue (int val) {
    if (n>SIZE)
        return;
    queue[tail] = val;
    tail=(tail+1)%SIZE;
    n++;
    return;
}
```

```
int dequeue (int *val) {
    if (n<=0)
        return;
    *val=queue[head];
    head=(head+1)%SIZE;
    n--;
    return;
}
```

enqueue and dequeue

- Operate on the different ends of the queue, using two variables tail and head
- Variable `n` is still shared

The problem is that increments and decrements of `n` can be lost

Critical section

The above examples are examples of critical sections in programs.

A Critical Section (CS) or Critical Region (CR) section of code, common to multiple processes (or Threads), in which they can access (read and write) shared objects.

In other words it is a section of code in which multiple processes (or threads) are competing for the use of shared resources.

If the conditions for a race condition (Sharing, Writing, Concurrency) are not present, then there is no Critical Section

Race conditions could be prevented if:

- No P (or T) executes in the same CS simultaneously
- No other P (or T) can execute, when a P (or T) executes in the CS
- The code in the CS is executed by a single P ((or T)) at a time
- The code in the CS is executed in mutual exclusion

Solution:

Establish an access protocol that enforces mutual exclusion for each CS.

Entering a CS, a thread executes a "reservation" code:

- The reservation code must block the P (or T) if another P (or T) is using its CS.

Leaving its CS, a P (or T) executes a code to release the CS

- The release possibly unlocks another P (or T) which was waiting in the "reservation" code of its CS

Access protocol

```
while (TRUE) {
    ...
    reservation code
    Critical Section
}
```

```

    release code
    ...
    non critical section
}

```

Every CS is protected by an

- enter code (reservation, or prologue)
- exit code (release, or epilogue)

Remember that non-critical sections should not be protected

Conditions

Each solution to the CS problem must match the following requirements.:

- Mutual exclusion
 - Only one P (or T) at a time must gain access to the CS
- Progress
 - If no P (or T) is in the CS, and a P (or T) wants to enter, it must be able to do it in a defined time
 - Only the P (or T) in the reservation phase can participate to the selection
 - No P (or T) outside the CS can block other P (or T)
 - That is, deadlock between P (or T) must be avoided
- Defined wait
 - There must be a maximum number of times in which other P (or T) can access the CS, before a specific P (or T) can access
 - That is we must avoid starvation of P (or T)
- Each solution should be symmetrical
 - The selection of the P (or T) that must access the CS should not depend on
 - Relative priority between P (or T)
 - Relative speed between P (or T)

Solutions

- Software functions
 - Solutions without special CPU instructions, which depend on the logic of an algorithm
- Hardware
 - Solutions based on special hardware characteristics, or special (atomic) CPU instructions
- System calls
 - The kernel provides the data structures, and the related system calls, that the programmer can properly use for solving the mutual exclusion problem

Software solutions

The software solutions to the CS problem are based on the use of shared (global) variables. This is available on systems with shared memory

We will analyze the solution with only two P (or T)

- They are named threads $P_i (T_i)$ and $P_j (T_j)$, $j = i - 1$

The proposed solution is not easily extended to more than two threads.

Mutual exclusion

Sol1

```

//Shared variables:
int flag[2]={FALSE, FALSE}

//P_i

```

```

while (TRUE) {
    while (flag[j]);
    flag[i] = TRUE;
    CS
    flag[i] = FALSE;
    non critical section
}

//P_j
while (TRUE) {
    while (flag[i]);
    flag[j] = TRUE;
    CS
    flag[j] = FALSE;
    non critical section
}

```

The problem is that both tasks can get passed the while at the same time (since `TRUE` isn't set yet), so both operate on the critical.

So the problems is that the lock variable is controlled and changed by two separate statements. So there is **no mutual exclusion**

Sol2

```

//Shared variables:
int flag[2]={FALSE, FALSE}

//P_i
while (TRUE) {
    flag[i] = TRUE;
    while (flag[j]);
    CS
    flag[i] = FALSE;
    non critical section
}

//P_j
while (TRUE) {
    flag[j] = TRUE;
    while (flag[i]);
    CS
    flag[j] = FALSE;
    non critical section
}

```

There is the chance of getting a **livelock** (deadlock), since both threads can set their flag to `TRUE` and wait forever

Sol3

```

# Shared variables
int turn =i //or int turn =j

//P_i
while (TRUE) {
    while (turn!=i);
    CS
    turn = j;
    non critical section
}

// P_j
while (TRUE) {
    while (turn!=j);
    CS
}

```

```

    turn = i;
    non critical section
}

```

The problem here is undefined wait:

- P_i and P_j access their CS only alternatively, so if P_i has no interest in using its CS, P_j cannot enter its CS (**starvation**)

Sol4

```

// Shared variables
int turn = i;
int flag[2] = {FALSE, FALSE};

//P_i
while (TRUE) {
    flag[i] = TRUE;
    turn = j;
    while (flag[j] && turn==j);
    CS
    flag[i] = FALSE;
    non critical section
}

//P_j
while (TRUE) {
    flag[j] = TRUE;
    turn = i;
    while (flag[i] && turn==i);
    CS
    flag[j] = FALSE;
    non critical section
}

```

All conditions met

Hardware solutions

Hardware solutions to the CS problem can be classified as follows:

- Solutions for systems that do not allow preemption
- Solutions for systems that allow preemption
 - Solutions based on interrupts management
 - Solutions based on an "extension" of software solutions, or based on
 - Some kind of lock
 - Some kind of atomic instruction

Systems without preemption

In mono-processor systems without preemption

- The CS problem does not exist, because only a P (or T) can use the only CPU at a certain time, and this P (or T) cannot be interrupted

However this situation rarely occurs because:

- Systems are often multi-processor or multicore, and even without preemption the parallelism is effective: i.e., distinct processors or cores can concurrently execute more than one P (or T)
- Kernels without preemption are not secure, have excessive response times, and are not suitable for "real-time"

Systems with preemption

In a system with preemption

- A running process can be interrupted
- As a matter of fact, the operating system or the arrival of an interrupt changes/preempts the control flow to another process
- The original process will be terminated later

In mono-processor system with preemption:

It is possible to solve CS problem with interrupts

- Disable interrupts in the reservation section
- Enable interrupts in the release section
 - Used only inside the kernel, and for short sections
 - In multi-processor (multi-core) the interrupts must be disabled on all processors

```
while (TRUE) {
    disable interrupt
    CS
    enable interrupt
    non critical section
}
```

Using the interrupt mechanism

In general, disabling interrupts has several disadvantages:

- The procedure is inherently insecure
 - What happens if to a user process is given the right to disable interrupts, and that process has an incorrect behavior?
 - This opportunity can be provided only to kernel level processes (super-user)
- In multi-processor (multi-core) systems it is necessary to disable the interrupt on all processors
 - The interrupt disabling request must be sent
 - Long processing times are needed
 - System management becomes non real-time

Using lock-unlock mechanisms

An alternative strategy is to simplify the software solutions, using locking mechanisms supported by the hardware.

A lock can be used to protect a CS.

The lock value allows or prohibits access to the CS.

It must be an "atomic" indivisible instruction executed in a single "memory cycle", which

- Cannot be interrupted
- Allows testing and simultaneous setting of a shared variable

Two main atomic lock instructions exist

- `char TestAndSet(char *lock);`
 - Sets to one and returns the previous value of a shared lock variable
 - Executed in a single indivisible cycle
- `void swap (char *v1, char *v2);`
 - Swaps the content of two variables, one of which is a shared lock
 - Executed in a single indivisible cycle

```
char TestAndSet (char *lock) {
    char val;
    val = *lock;
    *lock = TRUE; //Sets the lock to TRUE
    return val; //Returns the previous value of the lock
}
```

```
void swap (char *v1, char *v2) {
    char *tmp;
    *tmp = *v1;
    *v1 = *v2;
    *v2 = *tmp;
    return;
}
```

Use

```
char lock=FALSE;
while (TRUE) {
    while (TestAndSet (&lock)); // lock, if lock==TURE the CS is busy thus wait, if lock=FALSE, set
    lock=TRUE and enter CS
    CS
    lock = FALSE; // unlock
    Non critical section
}
```

The disadvantage is that the while consumes CPU cycles while it waits on the permissoin to enter CS

```
char lock=FALSE;
while (TRUE) {
    key = TRUE; //reserves the CS
    while (key==TRUE)
        swap (&lock, &key); // Lock, If lock==FALSE the CS is free, set key=FALSE, lock=TRUE, and
    enter the CS
    CS
    lock = FALSE; // Unlock
    non critical section
}
```

The disadvantage is still the spin-lock as above

Mutual exclusion without starvation

The previous techniques:

- ensure mutual exclusion
- ensure progress, avoiding the deadlock
- They do not ensure the definire waiting for a process, or they do not guarantee non-starvation
- Are symmetric

To avoid starvation:

- Previous solution must be extended

```
while (TRUE) {
    waiting[i] = TRUE;
    while (waiting[i] && TestAndSet (&lock));
    waiting[i] = FALSE;
    CS
    j = (i+1) % N;
    while ((j!=i) && (waiting[j]==FALSE))
        j = (j+1) % N;
    if (j==i)
        lock = FALSE;
    else
        waiting[j] = FALSE;
    non critical section
}
```

Conclusion on hardware solutions

Advantages:

- Can be used in multi-processor environments
- Easily extensible to N threads
- Easy to use from the software/user point of view
- Symmetric

Disadvantages:

- Not easy to implement at the hardware level
 - Need atomic operations on global variables
- Possible starvation
 - The selection of processes for entering the CS using busy-waiting is arbitrary, and managed by the processes and not by the OS
- Busy waiting on spin-lock
 - Waste of resources (i.e., CPU cycles) for waiting
 - In practice, busy-waiting is used only for very short waiting
- Priority inversion: a higher priority task is preempted by a lower priority task.
 - Consider two threads H and L, of high and low priority, respectively, accessing a resource in mutual exclusion.
 - L is in its CS, H is blocked outside until L exits its CS.
 - If a third thread M of medium priority becomes ready, it preempts L, thus L does not leave its CS promptly, causing H, the highest priority process, to remain blocked.
- A possible solution to this problem is to use the priority inheritance protocol
 - A process holding a lock automatically inherits the priority of the process with the higher priority waiting for the same lock

Semaphores

The previous solutions are not satisfactory, because

- software solutions are complex to use from the point of view of the programmer
- hardware solutions are difficult to implement from the point of view of the hardware designer

OSs provide more appropriate primitives called semaphores

Definition

A semaphore `S` is a shared structure including

- A counter
- A waiting queue, managed by the kernel
- Both protected by a lock

```
typedef struct semaphore_tag {  
    char lock; // Lock variable protects count  
            // and queue management  
    int cnt; // Counter  
    process_t *head; // Thread list  
} semaphore_t;
```

Operations on `S` are atomic

- Atomicity is managed by the OS
- It is impossible for two threads to perform simultaneous operations on the same semaphore

So a semaphore `S` is

- An integer shared variable

- Protected by the operating system
- Usable for mutual exclusion and synchronization

Manipulation functions

Typical operations on a semaphore S:

`init (S, k)` , Defines and initializes the semaphore S to the value k

`wait (S)` , Allows (in the reservation code) to obtain the access of the CS protected by the semaphore S

`signal (S)` . Allows (in the release code) to release the CS protected by the semaphore S

`destroy (S)` , Frees the semaphore S

init (S, k)

Defines and initializes semaphore S to value k

Two types of semaphores:

- Binary semaphores
 - The value of k is only 0 or 1
- Counting semaphores
 - The value of k is non negative

```
init (S, k) {
    alloc (S);
    S=k;
}
```

wait (S)

- If the counter value of S is negative or zero blocks the calling T/P
 - If S is negative, its absolute value |S| indicates the number of waiting threads
- Not to be confused with the wait system call used to wait for a child process

The counter is decremented at each call

```
wait (S) {
    if (S==0)
        block();
    else S--;
```

signal (S)utilizzo

- Increases the semaphore S
 - If S counter is negative or zero some T/P was blocked on the semaphore queue, and it can be wakeup
- Originally called V(), from the Dutch language "verhogen", i.e., "to increment"
- Not to be confused with system call signal that is used to declare a signal handler

```
signal (S) {
    if (blocked())
        wakeup();
    else S++;
```

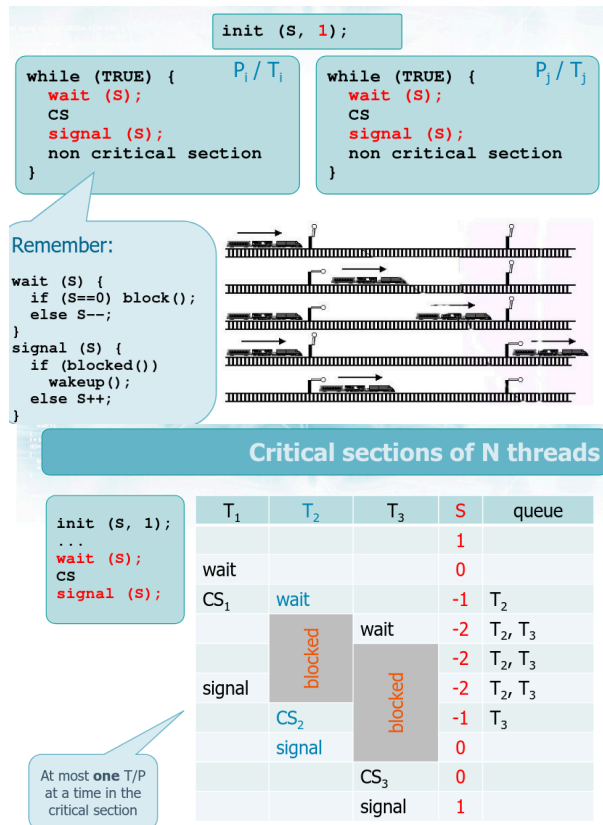
destroy (S)

Release semaphore S memory:

Actual implementations of a semaphore require much more of a simple global variable to define a semaphore. This function is often not used in the examples

```
destroy (S) {
    free (S);
}
```

Mutual exclusion with semaphores



Synchronization with semaphores

The use of semaphores is not limited to the Critical Section access protocol. They can be used to solve any synchronization problem using:

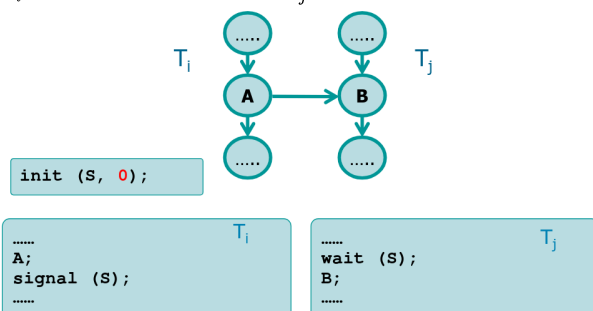
- An appropriate positioning of semaphores in the code
- Possibly, more than one semaphore
- Possibly, additional shared variables

Example 1

Pure synchronization

Obtain a specific order of execution:

- T_i executes code A before T_j executes code B

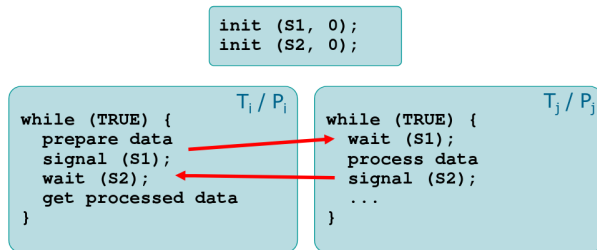


Example 2

Pure synchronization

Synchronize two T/P so that

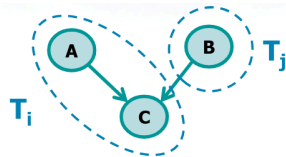
- T_j waits T_i
- then, T_i waits T_j
- It is a client-server schema



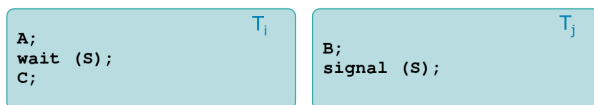
Example 3

Pure synchronization

Implement this precedence graph



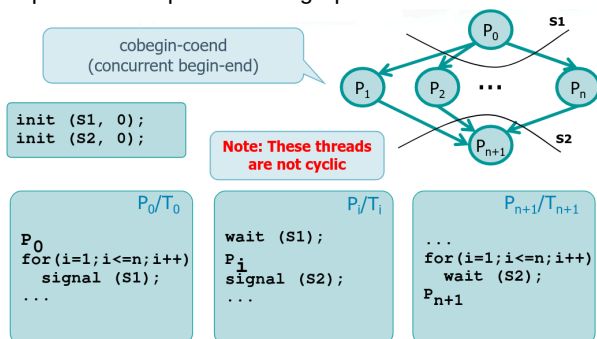
```
init (S, 0);
```



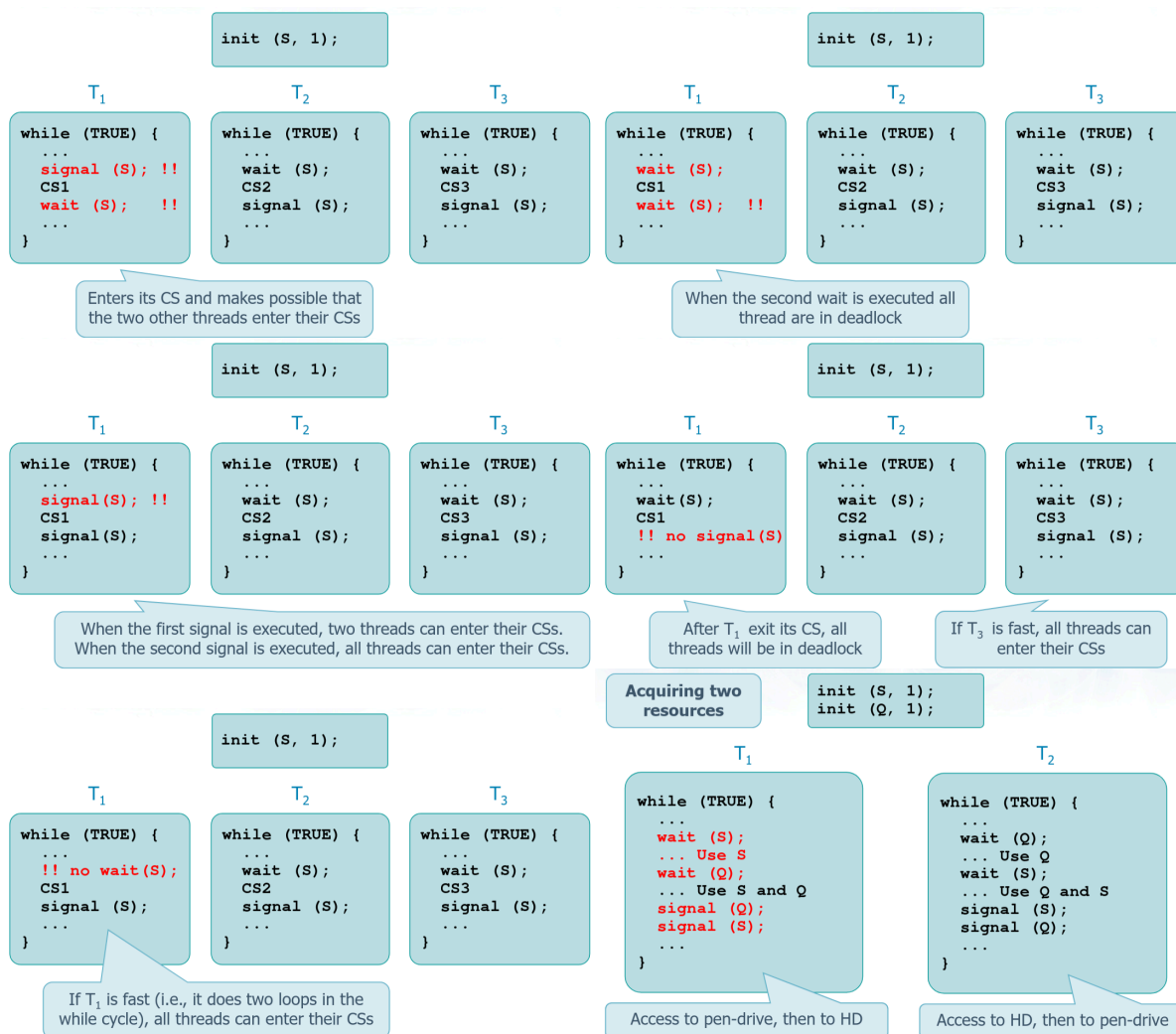
Example 4

Pure synchronization

Implement this precedence graph



Errors Examples



Implementation of a semaphore

Semaphores must be implemented without "active" busy waiting (spin-lock).

We define a semaphore as a C structure with

- A counter
- A list (queue) of processes

```
typedef struct semaphore_s {
    int cnt; // Number of processes
    process_t *head; // List of processes
} semaphore_t;
```

Therefor we have:

```
init (semaphore_t *S, int k) {
    alloc S;
    S->cnt = k;
    S->head = NULL;
}

wait (semaphore_t *S) {
    S->cnt--;
    if (S->cnt < 0) {
        push P to S->head;
        block P;
    }
}

signal (semaphore_t *S) {
    S->cnt++;
```

```

    if (S->cnt<=0) {
        pop P from S->head;
        wakeup P;
    }
}

destroy (semaphore_t *S) {
    while (S->cnt<=0) {
        free P from S->head;
        S->cnt++;
    }
}

```

The real implementation allows

- The counter
 - to have negative values. Its absolute value indicates the number of processes in the queue of the semaphore.
- The queue
 - Can be implemented with a pointer in the Process Control Block (PCB) of the processes
 - It uses the policies defined by the scheduler (e.g., FIFO)

Real implementation

There are several semaphores implementations

- Semaphores by means of a pipe
- POSIX Pthread
 - Condition variables (`pthread_cond_init`, `pthread_cond_wait`, `pthread_cond_signal` ...)
 - Semaphores
 - The most important
 - Mutex (for mutual exclusion)
- Linux semaphores (`semget`, `semop`, `semctl`)

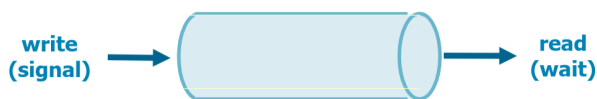
Notice that semaphores are

- Global shared objects (see `sem_init`)
- They are allocated by a thread, but they are kernel objects

By mean of a pipe

Given a pipe

- The counter of a semaphore is achieved by means of tokens
- Signal implemented using the write system call to write a token on the pipe (non-blocking)
- Wait implemented using the read system call to read a token from the pipe (blocking)



```

#include <unistd.h>
void semaphoreInit (int *S, int k) {
    char ctr = 'X';
    int i;
    if (pipe (S) == -1) {
        printf ("Error"); exit (-1);
    }
    for(i=0; i<k; i++)
        if (write(S[1], &ctr, sizeof(char)) != 1) {
            printf ("Error"); exit (-1);
        }
    return;
}

```

```

#include <unistd.h>
void semaphoreSignal (int *S) {
    char ctr = 'X';
    if (write(S[1], &ctr, sizeof(char)) != 1) {
        printf ("Error");
        exit (-1);
    }
    return;
}

#include <unistd.h>
void semaphoreWait (int *S) {
    char ctr;
    if (read (S[0], &ctr, sizeof(char)) != 1) {
        printf ("Error");
        exit (-1);
    }
    return;
}

```

Example:

```

int main() {
    int S[2];
    pid_t pid;
    semaphoreInit (S, 0);
    pid = fork();
    // Check for correctness
    if (pid == 0) { // child
        semaphoreWait (S);
        printf("Wait done.\n");
    }
    else { // parent
        printf("Sleep 3s.\n");
        sleep (3);
        semaphoreSignal (S);
        printf("Signal done.\n");
    }
    return 0;
}

```

POSIX semaphores

There are two types of POSIX semaphores

- Unnamed semaphores
 - Implemented in the internal memory of the process
 - They are used for the synchronization of threads within the same process
- Named semaphores
 - Implemented using shared memory, they are “process-shared semaphore”
 - They are generally used in the synchronization between processes
 - The name (sem_open) allows their use in different processes

We will be analyzing only the unnamed semaphores which:

- The implementation is independent from the OS, and it is defined in the semaphore.h header file
- Insert in the .c file: `#include <semaphore.h>`

The semaphore is a variable of type `sem_t`. A semaphore can be allocated statically or dynamically (`sem_t *sem1, *sem2...`)

Functions defined on semaphores:

- are named `sem_*`

- return -1 or error

```
int sem_init (sem_t *sem, int pshared, unsigned int value);
```

pshared value identifies the type of semaphore:

- If equal to 0, the semaphore is local to the threads of current process
- Otherwise, the semaphore can be shared between different processes (parent that initializes the semaphore and its children)

```
int sem_wait(sem_t *sem);
```

Standard wait

- If the semaphore is equal to 0, it blocks the caller until it can decrease the value of the semaphore

```
int sem_trywait(sem_t *sem);
```

Non-blocking wait

- If the semaphore counter has a value greater than 0, perform the decrement, and returns 0
- If the semaphore is equal to 0, returns -1 (instead of blocking the caller as `sem_wait` does)

```
int sem_post(sem_t *sem);
```

Standard signal

- Increments the semaphore counter, or wakes up a blocked thread if present

```
int sem_getvalue(sem_t *sem, int *valP);
```

NOT TO BE USED:

"The value of the semaphore may already have changed by the time `sem_getvalue()` returns." From the Linux manual.

Allows obtaining the value of the semaphore counter

- The value is assigned to `*valP`
- If there are waiting threads
 - 0 is assigned to `*valP` (Linux)
 - or a negative number whose absolute value is equal to the number of processes waiting (POSIX)

```
int sem_destroy(sem_t *sem);
```

Destroys the semaphore at the address pointed by sem

- Destroying a semaphore that other threads are currently blocked on produces undefined behavior (on error, -1 is returned)
- Using a semaphore that has been destroyed produces undefined results, until the semaphore has been reinitialized

Example:

```
#include "semaphore.h"
sem_t sem;
sem_init (&sem, 0, 0);

... create threads ...

sem_destroy (&sem);
```

would be:

```
sem_wait (&sem);

... SC ...

sem_post (&sem);
```

Example:

```
include "semaphore.h"
sem_t *sem;
sem = (sem_t *)
malloc(sizeof(sem_t));
sem_init (sem, 0, 0);

... create threads ...

sem_destroy (sem);
```

would be:

```
sem_wait (sem);

... SC ...

sem_post (sem);
```

Pthread mutex

Binary semaphores (mutex)

- A mutex is of type `pthread_mutex_t`
- System calls
 - `pthread_mutex_init`
 - `pthread_mutex_lock`
 - `pthread_mutex_trylock`
 - `pthread_mutex_unlock`
 - `pthread_mutex_destroy`

They can only assume 0 or 1 as values

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
```

Initializes the mutex referenced by `mutex` with attributes specified by `attr` (default= `NULL`)

- Return value
- 0 on success
- Error code otherwise

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

Control the value of mutex and

- Blocks the caller if the mutex is locked
- Acquire the mutex lock if the mutex is unlocked

Return value

- 0 on success
- Error code otherwise

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
```

- Similar to `pthread_mutex_lock`, but returns without blocking the caller if the mutex is locked

Return value

- 0 if the lock has been successfully acquired
- EBUSY error if the mutex was already locked by another thread

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- Release the mutex lock (typically at the end of a Critical Section)
- Return value
 - 0 on success
 - Error code otherwise

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- Free mutex memory
- The mutex cannot be used any more
- Return value
 - 0 on success
 - Error code otherwise

UNIT 5 AND 6 ARE MISSING SINCE THEY ARE EXERCISE RELATED